



Análisis de Datos II: sistemas expertos y redes de conocimiento

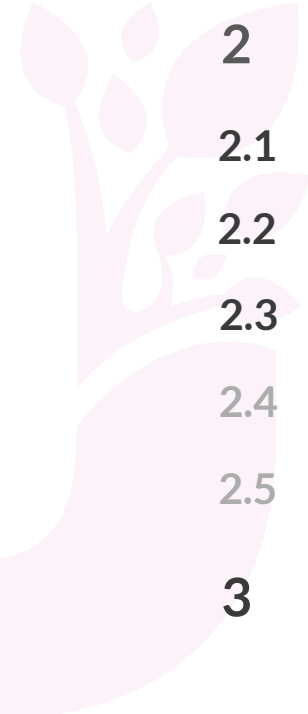
Clase 4
23 de abril de 2026

Prof. Dr. Agustín Asuaje



Sistemas Expertos

Temas de la clase

- 
- 1 Representación del conocimiento (*continuación*)
 - 2 Introducción a Sistemas Expertos
 - 2.1 Base de conocimientos (RBES)
 - 2.2 Motor de inferencia (RBES)
 - 2.3 Otros componentes
 - 2.4 Otros tipos de Sistemas Expertos
 - 2.5 Shells
 - 3 Pautas para el Trabajo Práctico

Temas de la clase

- 
- 1 Representación del conocimiento (*continuación*)
 - 2 Introducción a Sistemas Expertos
 - 2.1 Base de conocimientos (RBES)
 - 2.2 Motor de inferencia (RBES)
 - 2.3 Otros componentes
 - 2.4 Otros tipos de Sistemas Expertos
 - 2.5 Shells
 - 3 Pautas para el Trabajo Práctico



RC vs Bases de Datos



	Base de Datos (BD)	Representación del Conocimiento (KR)
Propósito	Almacenar y recuperar datos de forma estructurada.	Representar, razonar y extraer conclusiones a partir de conocimiento.
Formato	Tablas, documentos, archivos estructurados.	Reglas, ontologías, grafos semánticos, lógica formal.
Tipo de Información	Datos explícitos (hechos).	Hechos + Relaciones + Reglas de inferencia.
Ejemplos	MySQL, MongoDB, PostgreSQL.	Ontologías en OWL, Sistemas basados en reglas, Redes Semánticas.

IA Simbólica: Representa el conocimiento mediante **símbolos, reglas lógicas y estructuras explícitas** (ej. sistemas expertos, redes semánticas, marcos, etc). Es interpretable, pero poco flexible para aprender de datos nuevos.

IA Subsimbólica: Se basa en **redes neuronales y modelos estadísticos** sin usar reglas explícitas. Aprende patrones automáticamente, pero es difícil de explicar (Machine Learning).

Principales formas de Representación del Conocimiento en Deep Learning

Tipo de Representación	Descripción	Ejemplo de Modelos	Ventajas	Desventajas
Representaciones Distribuidas (Embeddings)	Representación de conceptos como vectores en espacios de alta dimensión.	Word2Vec, GloVe, FastText, BERT, GPT	Captura relaciones semánticas automáticamente.	No tiene lógica explícita ni razonamiento.
Redes Neuronales como Modelos Implícitos de Conocimiento	El conocimiento se almacena en los pesos de las conexiones neuronales.	GPT-4, AlphaGo, ResNet	Aprende patrones sin intervención humana.	Caja negra (difícil de interpretar).
Modelos de Atención y Transformers	Mecanismo que permite enfocarse en partes clave de la entrada para capturar relaciones.	BERT, GPT, T5	Captura contexto y significado mejor que los embeddings simples.	No puede hacer razonamiento lógico profundo.
Knowledge Graph Embeddings (Híbrido)	Convierte entidades y relaciones de un grafo en vectores en un espacio continuo.	Google Knowledge Graph, DBpedia Embeddings	Combina estructura de grafos con flexibilidad de Deep Learning.	Depende de datos estructurados iniciales.

IA simbólica vs subsimbólica



Característica	IA Simbólica (Sistemas Expertos)	Aprendizaje Automático (Deep Learning, Redes Neuronales)
Interpretabilidad	Alta (las reglas son explícitas)	Baja (modelo de caja negra)
Razonamiento	Fuerte (puede seguir pasos lógicos)	Débil (aprende correlaciones, no causalidad)
Aprendizaje a partir de datos	Débil (necesita actualización manual de reglas)	Fuerte (se adapta automáticamente)
Escalabilidad	Baja (difícil de mantener con muchas reglas)	Alta (escalable con grandes volúmenes de datos)
Sentido común	Bueno (si se codifica explícitamente)	Deficiente (a menudo carece de lógica del mundo real)
Manejo del ruido	Débil (sensible a errores)	Fuerte (robusto frente a datos ruidosos)



IA simbólica y subsimbólica ¿combinadas?



La **IA Híbrida** combina **IA Simbólica** (basada en reglas y lógica) con **IA Subsimbólica** (aprendizaje automático, redes neuronales).

Su objetivo es aprovechar las fortalezas de ambas:

- **Razonamiento estructurado y explicable** (IA Simbólica).
- **Aprendizaje automático y reconocimiento de patrones** (IA Subsimbólica).



Ejemplos

Traducción Automática (Google Translate, DeepL)

- Usa redes neuronales (Transformers, como GPT o BERT) para traducir frases con contexto.
- Aplica gramática y reglas lingüísticas (IA simbólica) para mejorar la sintaxis y la estructura.
- El aprendizaje automático generaliza bien, pero las reglas lingüísticas evitan errores gramaticales o traducciones sin sentido.



Ejemplos

Sistemas de Diagnóstico en Salud

- Usa redes neuronales (**IA subsimbólica**) para analizar imágenes médicas.
- Complementa con sistemas expertos médicos (**IA simbólica**) que siguen protocolos de diagnóstico.
- Las imágenes médicas pueden ser interpretadas por redes neuronales, pero el diagnóstico necesita ser explicado con reglas simbólicas para confianza médica.

Vehículos Autónomos (Tesla, Waymo, Cruise)

- Redes neuronales procesan imágenes y detectan peatones, semáforos, otros autos.
- Modelos simbólicos aplican reglas de tránsito y planificación de rutas.
- Deep Learning reconoce objetos en el entorno, pero la conducción sigue reglas fijas que deben cumplirse (ej. detenerse en rojo).



IA híbrida - Sistemas Neuro-Simbólicos



Componente	Contenido Visual sugerido
Título: Sistemas Neuro-Simbólicos: Lo mejor de dos mundos Puntos Clave: • IA Simbólica (Clásica): Determinista, explicable, basada en reglas rígidas. • IA Subsimbólica (Neuronal): Probabilística, intuitiva, manejo de lenguaje natural. • La Sinergia: El LLM actúa como interfaz y extractor de hechos; el Motor de Reglas valida y ejecuta la lógica final.	Un diagrama que muestre una "Caja Neuronal" (LLM) procesando lenguaje natural y enviando "Hechos Estructurados" a una "Caja Simbólica" (Motor de Inferencia). <i>Tag sugerido: [Image of Neuro-symbolic AI architecture combining LLMs and Rule Engines]</i>



IA híbrida - metodologías



Método	Cómo combina IA simbólica y subsimbólica	Ejemplo
Acoplado (Loose Coupling)	Se combinan resultados de ambos métodos sin integración profunda	Chatbots, sistemas de búsqueda
Integrado (Tight Coupling)	Los métodos simbólicos y subsimbólicos trabajan juntos en el mismo sistema	IBM Watson, análisis financiero
Neuro-Simbólico (Fusionado)	La red neuronal aprende reglas y razona con ellas	AlphaCode, robótica avanzada

Temas de la clase

- 
- 1 Representación del conocimiento (*continuación*)
 - 2 **Introducción a Sistemas Expertos**
 - 2.1 Base de conocimientos (RBES)
 - 2.2 Motor de inferencia (RBES)
 - 2.3 Otros componentes
 - 2.4 Otros tipos de Sistemas Expertos
 - 2.5 Shells
 - 3 Pautas para el Trabajo Práctico

¿Qué es un Sistema Experto?



“Un sistema experto es un programa de computadora inteligente que usa conocimiento y procedimientos de inferencia para resolver problemas que normalmente requieren la experiencia humana.” Edward Feigenbaum (1982)



Creador de:

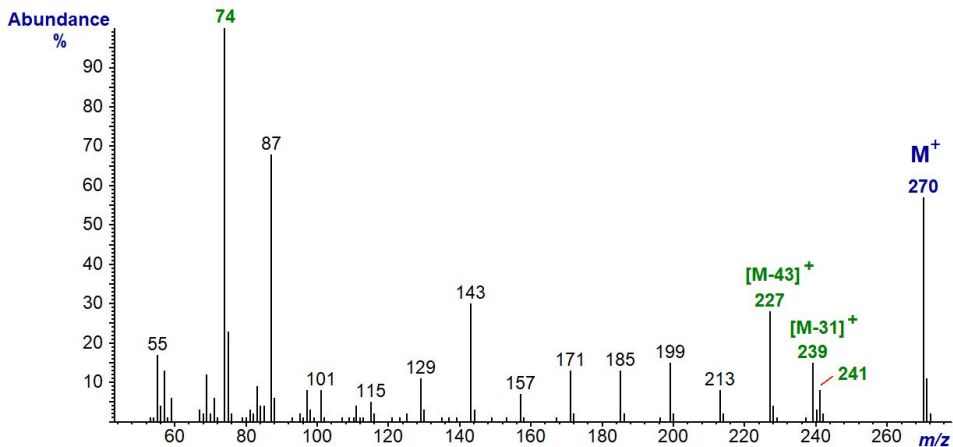
- DENDRAL (1965)
- MYCIN (1972)



Sistemas Expertos (ejemplos)



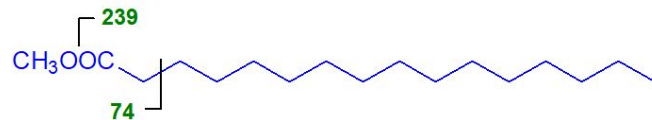
DENDRAL (1965)



Espectro de masas



E. Feigenbaum en la Universidad de Stanford





Sistemas Expertos (ejemplos)

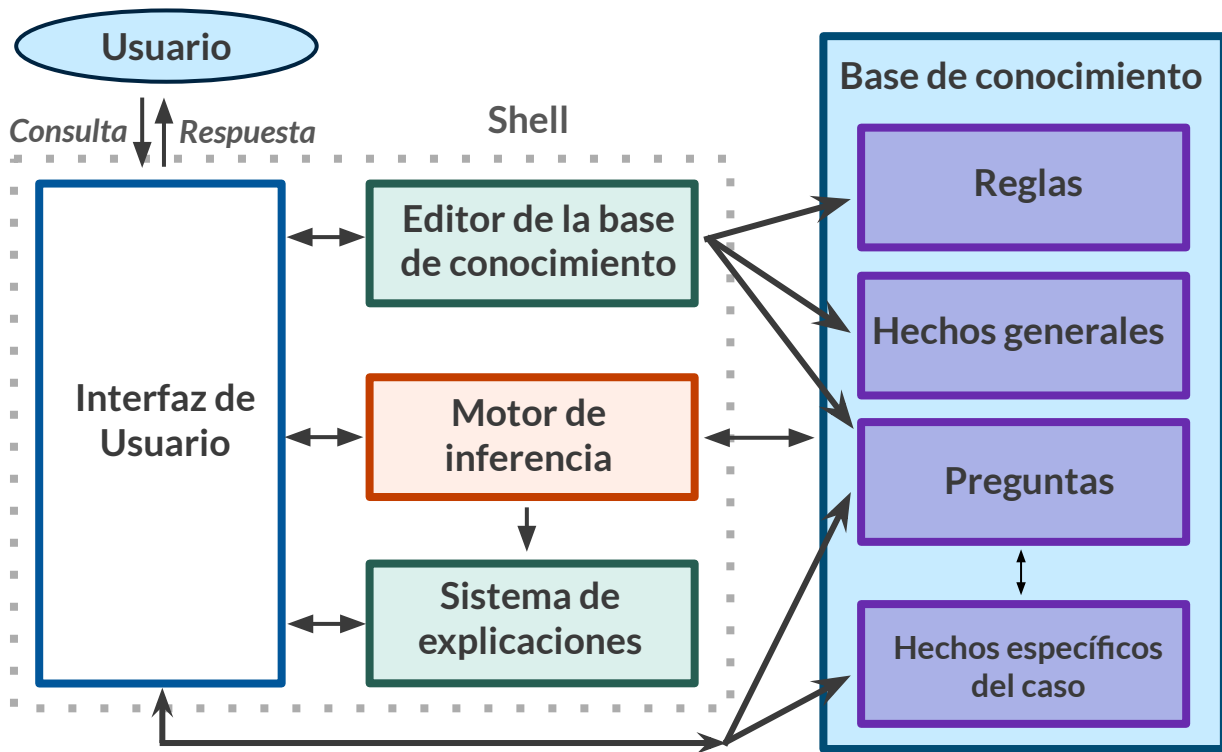


XCON (eXpert **CON**figurer) fue un sistema experto desarrollado por Digital Equipment Corporation (DEC) en los **años 80** para ayudar a los ingenieros a configurar sistemas informáticos (hardware) complejos según los pedidos de los clientes.

Se estima que ahorró a DEC 40 millones de dólares anuales en costos operativos.



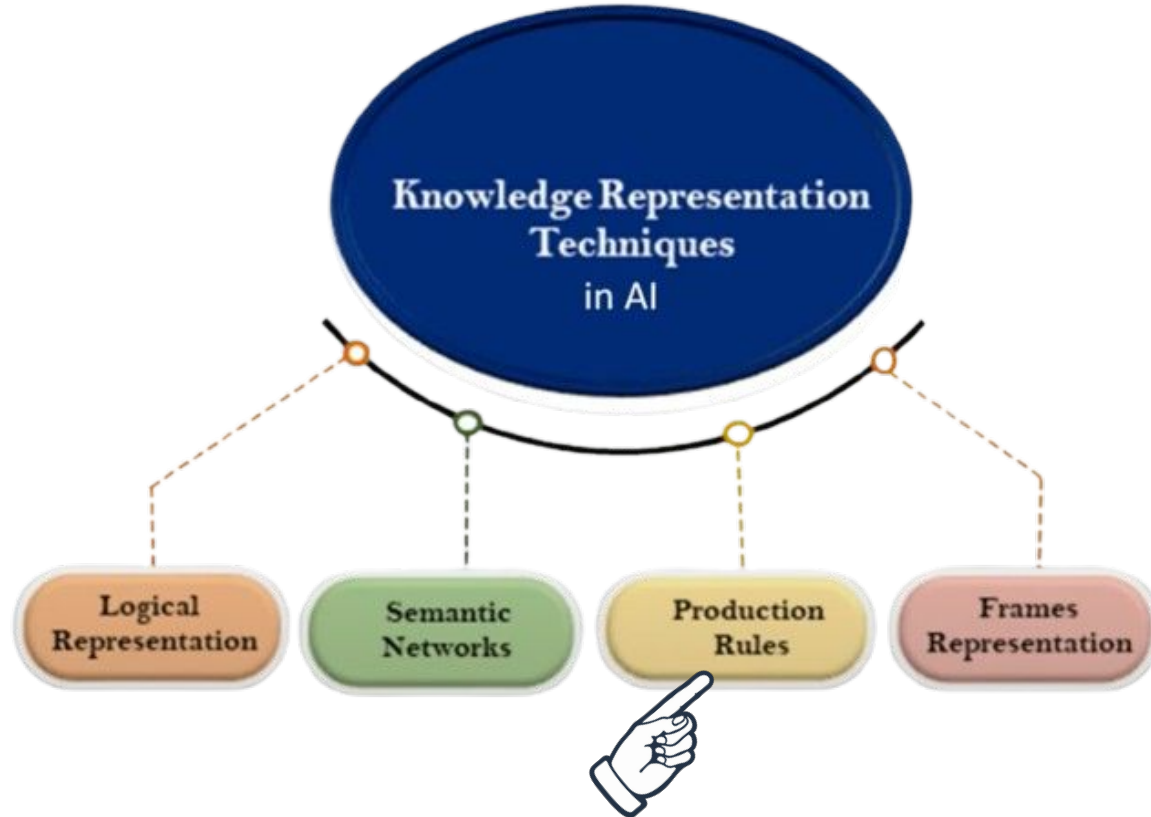
Elementos de un Sistema Experto (RBES)



RBES = Rule Based Expert System



Representación del Conocimiento





Representación con Reglas (repaso)



La representación del conocimiento en forma de “reglas”, es decir, expresiones SI-ENTONCES (IF-THEN):

Expresión **condicional-conclusión**:

SI <condición(es)> ENTONCES <conclusión>

Expresión **condicional-acción**:

SI <condición(es)> ENTONCES <acción>

Equivalente a implicaciones proposicionales o de lógica de primer orden (FOL).

Ejemplo:

SI el paciente tiene altos niveles de la enzima ferritina en su sangre

Y el paciente tiene la mutación Cys282→Tyr en el gen HFE

ENTONCES concluir que el paciente tiene hemocromatosis.

Sinónimos

Condición: premisa, antecedente, variable independiente.

Conclusión: acción, resultado, consecuencia, meta, variable dependiente.

Temas de la clase

- 
- 1 Representación del conocimiento (*continuación*)
 - 2 **Introducción a Sistemas Expertos**
 - 2.1 Base de conocimientos (RBES)
 - 2.2 Motor de inferencia (RBES)
 - 2.3 Otros componentes
 - 2.4 Otros tipos de Sistemas Expertos
 - 2.5 Shells
 - 3 Pautas para el Trabajo Práctico



Base de conocimientos (RBES)



- Almacena la experiencia y el conocimiento que el sistema puede utilizar.
- También conocida como memoria de producción (en sistemas de producción).
- Se desarrolla de manera específica para una tarea experta concreta.
- Existe de forma independiente del shell del sistema experto.



Base de conocimientos (RBES)



Componentes

Base de Reglas:

- Conjunto de **reglas de implicación** que capturan el conocimiento del dominio.

Base de Hechos (Memoria a largo plazo):

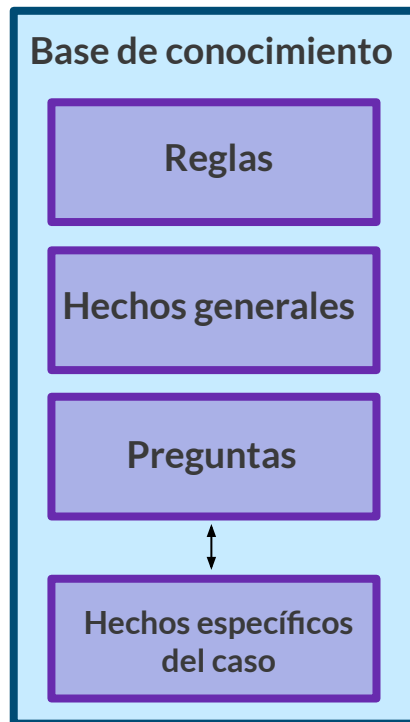
- Contiene **hechos generales que no cambian**.
- Puede hacer referencia a una **base de datos** con información clave o valores de variables.

Base de Preguntas:

- Aquí se pueden especificar **preguntas en lenguaje natural**.
- Se usa para solicitar **hechos específicos del caso** al usuario.
- El **motor de inferencia** usa las respuestas para determinar si una regla debe activarse.

Hechos Específicos del Caso (Memoria a corto plazo):

- Son los **hechos proporcionados por el usuario** para una consulta determinada.
- Pueden ingresarse de dos maneras: Como respuestas a preguntas (cuando se requieren pocos hechos) o Cargados en bloque (cuando se necesitan muchos hechos únicos o ya están disponibles).





Base de reglas (RBES)



Un conjunto “**mayormente**” **desordenado** de reglas **no repetitivas**.

Dependiendo de la complejidad de la tarea:

- En problemas del mundo real, la base de reglas puede contener **desde unos cientos hasta miles de reglas**.

Uso de reglas IF-AND-THEN y manejo de ‘OR’

Ejemplo de cómo manejar una regla con OR:

- **Regla original:**
- **SI A O B O C ENTONCES D**

Para implementarla, se descompone en **reglas individuales**:

- **SI A ENTONCES D**
- **SI B ENTONCES D**
- **SI C ENTONCES D**

Ejemplo de anotación de una regla

```
SI son_of($hijo, $padre)  
ENTONCES father_of($padre, $hijo)
```

(Esto significa que si alguien es hijo de una persona, entonces esa persona es su padre.)



Base de hechos generales (RBES)



- Un conjunto “mayormente” desordenado de hechos que asignan valores a proposiciones o declaraciones de predicados.
- Los hechos deben seguir la misma sintaxis que las reglas para que el motor de inferencia pueda hacer coincidencias y aplicar reglas.

Ejemplo de Regla:

```
SI son_of($hijo, $padre)  
ENTONCES father_of($padre, $hijo)
```

Ejemplo de Hechos:

```
son_of(Steve, Joe)  
son_of(Bill, Dave)
```

(Esto significa que Steve es hijo de Joe, y Bill es hijo de Dave.)



Base de Preguntas (RBES)



Conjunto “mayormente” desordenado de preguntas que el Sistema Experto (ES) aplica cuando es necesario para complementar la base de conocimiento con hechos específicos del caso tratado.

Tipos de respuestas que puede recibir:

- Sí/No, Verdadero/Falso, Opción múltiple, Ingresar un valor (entero, decimal, texto).

Ejemplo de pregunta:

who_father(\$ans)

¿Quién es tu padre?

\$ans = select_1

1: Joe

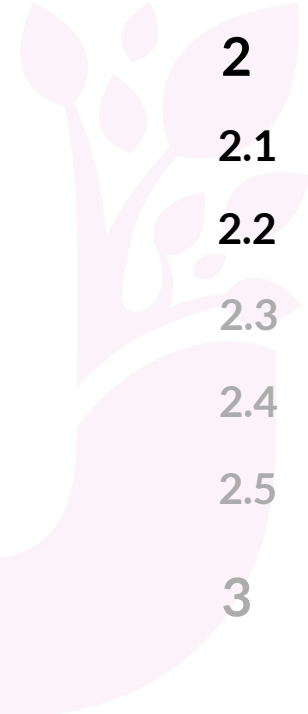
2: Steve

3: Bill

4: Dave

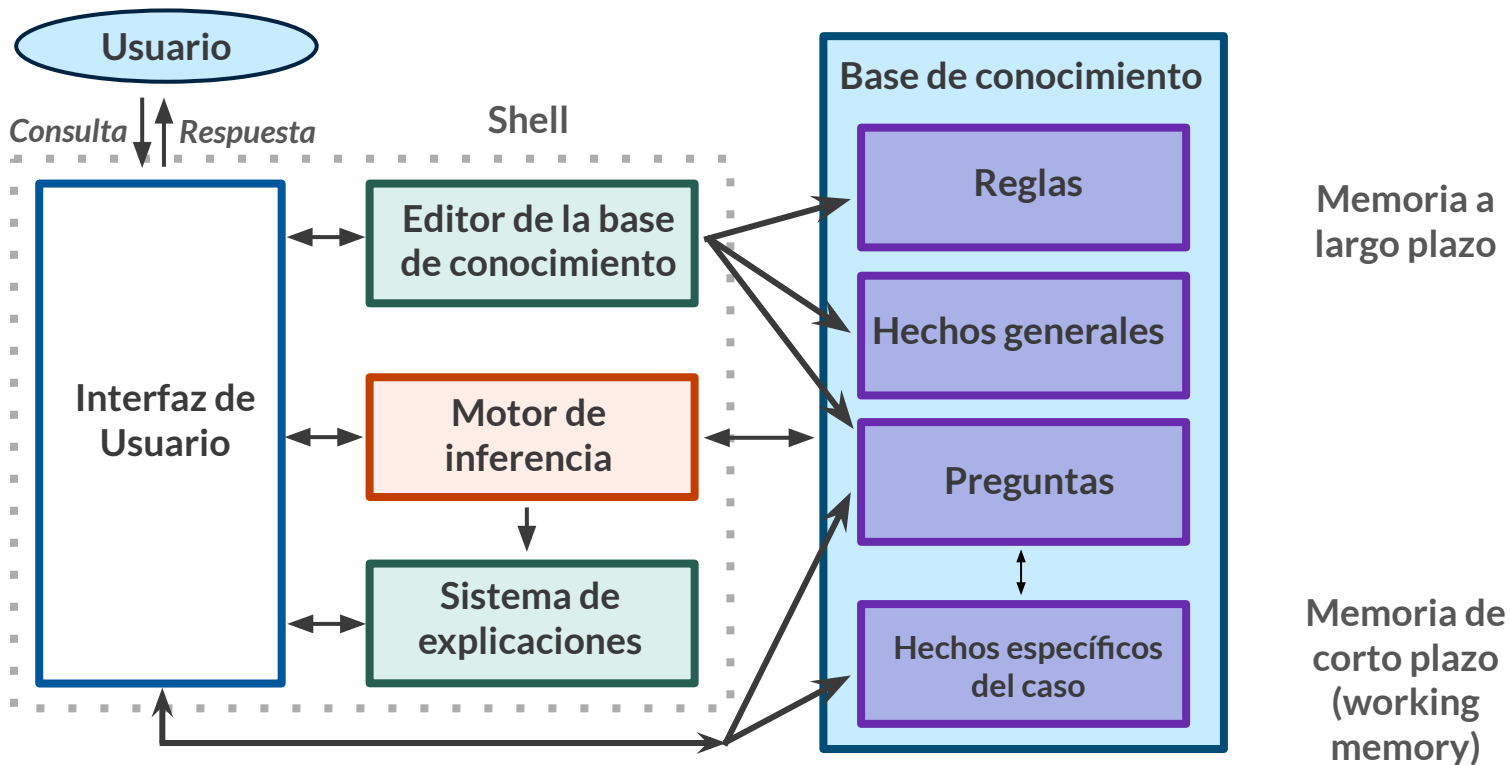
5: No lo sé

Temas de la clase

- 
- 1 Representación del conocimiento (*continuación*)
 - 2 **Introducción a Sistemas Expertos**
 - 2.1 Base de conocimientos (RBES)
 - 2.2 Motor de inferencia (RBES)
 - 2.3 Otros componentes
 - 2.4 Otros tipos de Sistemas Expertos
 - 2.5 Shells
 - 3 Pautas para el Trabajo Práctico



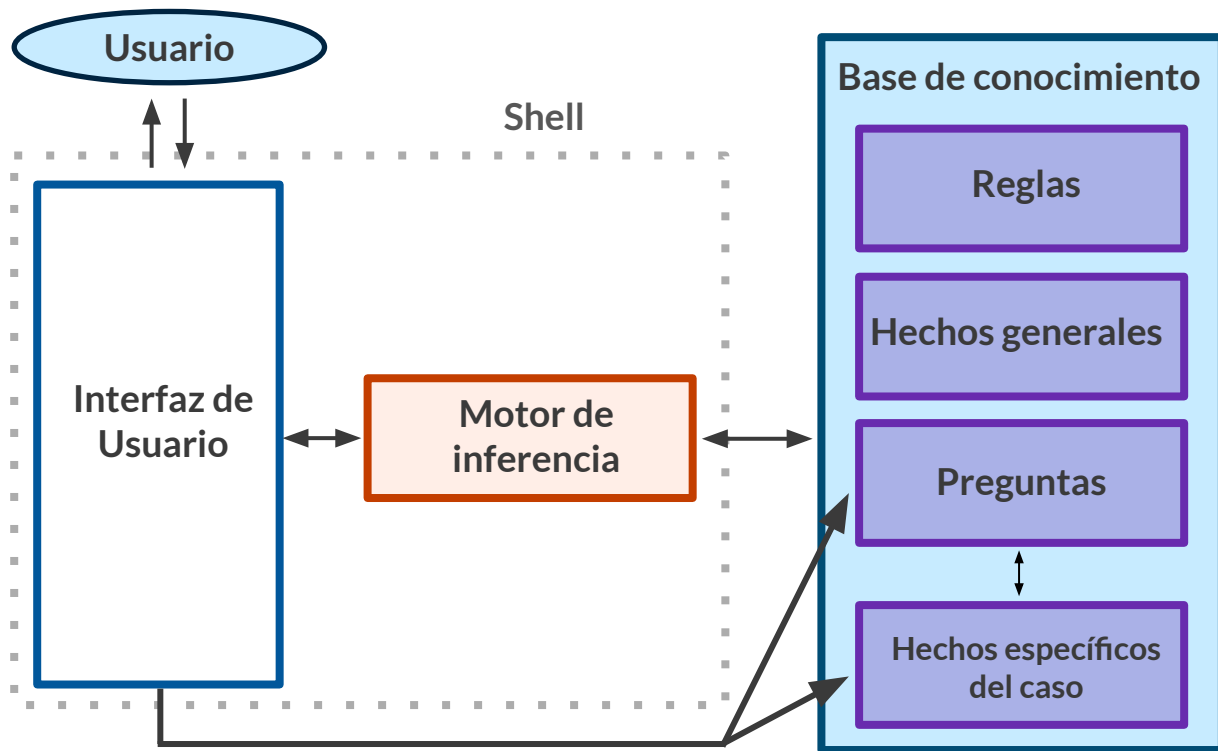
Elementos de un Sistema Experto (RBES)



RBES = Rule Based Expert System



Motor de inferencia (RBES)





Motor de inferencia (RBES)



- También conocido como **intérprete de reglas**.
- Es la “**parte pensante**” o el “**cerebro**” del sistema experto.
- Controla **cómo se aplican las reglas IF-THEN** a los hechos.

Puede usarse para:

- Llegar a una **hipótesis o solución**.
- Refinar una **hipótesis o solución** existente.
- Resolver **conflictos** entre múltiples hipótesis o soluciones en competencia.

Enlace entre reglas y hechos:

- **Genera nuevos hechos** a partir de la combinación de reglas y hechos existentes.
- Los **nuevos hechos representan conclusiones** sobre el estado del dominio basándose en las observaciones.



Motor de inferencia (RBES)



Ciclo de
Coincidencia-Resolución -
Acción
(Match-Resolve-Act cycle)





Coincidencia de Patrones (*Pattern matching*)



1

Coincidencia
de Patrones

- La coincidencia determina el conocimiento aplicable en una situación dada (es la **parte de “búsqueda”** del sistema).
- Las reglas se “activan” (**se disparan**) según el tipo de inferencia o encadenamiento que se utilice.

Encadenamiento hacia adelante (Forward Chaining):

- Se comparan los antecedentes de las reglas con la base de hechos.

Encadenamiento hacia atrás (Backward Chaining):

- Se comparan los consecuentes de las reglas con la base de hechos y la base de reglas.

Importante:

- La comparación de muchas reglas contra muchos hechos puede ser computacionalmente costosa (salvo uso de algoritmos tipo RETE).
- Las reglas que coinciden se añaden a la “**agenda**”, que es una lista priorizada de reglas generada por el motor de inferencia.



Vinculación de Variables (*Variable Binding*)



- Las variables en las reglas adquieren valores durante el proceso de coincidencia (matching).
- Tanto los antecedentes como los consecuentes pueden contener variables.
- Por ahora, asumimos que las variables tienen el formato (\$x).

Ejemplo de Regla con Variables:

SI son_of(\$son, \$father)

ENTONCES father_of(\$father, \$son)

Hechos:

son_of(Steve, Joe)

daughter_of(Mary, Joe)

Supongamos Encadenamiento Hacia Adelante La regla coincide con el hecho son_of(Steve, Joe)

Se realiza la vinculación de variables:

\$son = Steve

\$father = Joe

Cuando se activa el consecuente de la regla se agrega father_of(Joe, Steve) a los hechos específicos del caso (memoria de trabajo).



Motor de inferencia (RBES)



Ciclo de
Coincidencia-Resolución -
Acción
(Match-Resolve-Act cycle)





2

Resolución de Conflictos

La coincidencia de patrones a menudo produce más de una regla aplicable, que se añaden a la agenda.

¿Cuál ejecutar primero? ¿O solo queremos que se active una?

Ejemplo:

- Podríamos querer que solo se prescriba un medicamento.
- Si asumimos que solo queremos que una acción se active, entonces las instancias de reglas están en conflicto.

Existen diferentes estrategias para resolver estos conflictos según la aplicación.



2

Resolución
de Conflictos

Estrategias

Orden (Order) – Orden en el archivo fuente:

- Selecciona la primera regla que coincida en la base de reglas.
- No es ideal para conjuntos de reglas muy grandes.

Especificidad (Specificity):

- Selecciona la regla más específica.
- Si las condiciones de una regla son un subconjunto de otra, elige la más específica.
- **Ejemplo:** (SI A Y B ENTONCES C) vs. (SI A ENTONCES D)
→ Se elige la primera.

Orden Léxico (Lexical Order):

- Se basa en orden alfabético u orden de diccionario.
- Algo arbitrario, pero útil en algunos dominios.



2

Resolución
de Conflictos

Estrategias

Importancia (*Saliency*)

- Selecciona la regla con mayor prioridad.
- Las reglas se definen con un puntaje de prioridad.
- Desaconsejado en bases de reglas muy grandes, ya que es difícil de configurar con precisión.

Ejemplo:

Si el paciente tiene dolor,
ENTONCES recetar analgésicos (**prioridad 10**)

Si el paciente tiene dolor en el pecho,
ENTONCES derivar a urgencias (**prioridad 100**)



2

Resolución
de Conflictos

Estrategias

Recencia (Recency)

- Selecciona la regla añadida o modificada más recientemente.
- También puede elegir la regla cuyos antecedentes fueron modificados más recientemente.
- Requiere etiquetas de tiempo ("time-tags") para rastrear cambios.

Refactorización (Refactoriness)

- Evita que la misma instancia de vinculación de una regla se dispare más de una vez.



Motor de inferencia (RBES)



Ciclo de
Coincidencia-Resolución -
Acción
(Match-Resolve-Act cycle)



3

Activación de la
Consecuencia

Agenda



Hechos
específicos
del caso

Una vez que se selecciona una regla de la agenda...

Activación/Ejecución/Disparo de la Regla:

- Ejecutar el consecuente de la regla seleccionada (Encadenamiento hacia adelante).
- El nuevo hecho resultante se añade a los hechos específicos del caso.
- Se elimina la regla ejecutada de la agenda.

¿Cuándo termina el ciclo?

- Cuando no quedan más reglas en la agenda.
- O cuando se encuentra un comando de parada explícito.



Reglas de inferencia (RBES)



Encadenamiento hacia adelante (*forward chaining*) – Basado en datos

- Se inicia con hechos, se determinan las reglas aplicables y se aplica una.
- Ideal para: Pronóstico, Monitoreo, Control
- Ejemplo: “¿Qué enfermedades podría tener un paciente con fiebre y dolor de cabeza?”

Encadenamiento hacia atrás (*backward chaining*) – Basado en objetivos

- Se buscan reglas que descomponen el objetivo en subobjetivos más pequeños.
- Ideal para: Problemas de diagnóstico
- Ejemplo: “¿Tiene neumonía el paciente?”

¿Cuál es mejor?

Depende de la aplicación y del problema que se quiera resolver.



Fw vs Bw chaining (RBES)



Encadenamiento hacia adelante	Encadenamiento hacia atrás
Razonamiento basado en datos	Razonamiento basado en objetivos
Comienza con hechos en la base de conocimiento	Comienza con un objetivo ingresado por el usuario que debe probarse
Las reglas de inferencia se aplican haciendo coincidir los hechos con la parte “ SI ”(IF) de las reglas en la base de conocimiento	Las reglas de inferencia se aplican haciendo coincidir el objetivo con la parte “ ENTONCES ” (THEN) de las reglas en la base de conocimiento
Da como resultado nuevos hechos añadidos a la base de conocimiento	Da como resultado subobjetivos que deben probarse
Las reglas se ejecutan automáticamente cuando se activa el encadenamiento hacia adelante para inferir nuevos hechos	Las reglas se usan directamente para probar un objetivo



Encadenamiento hacia adelante con reglas

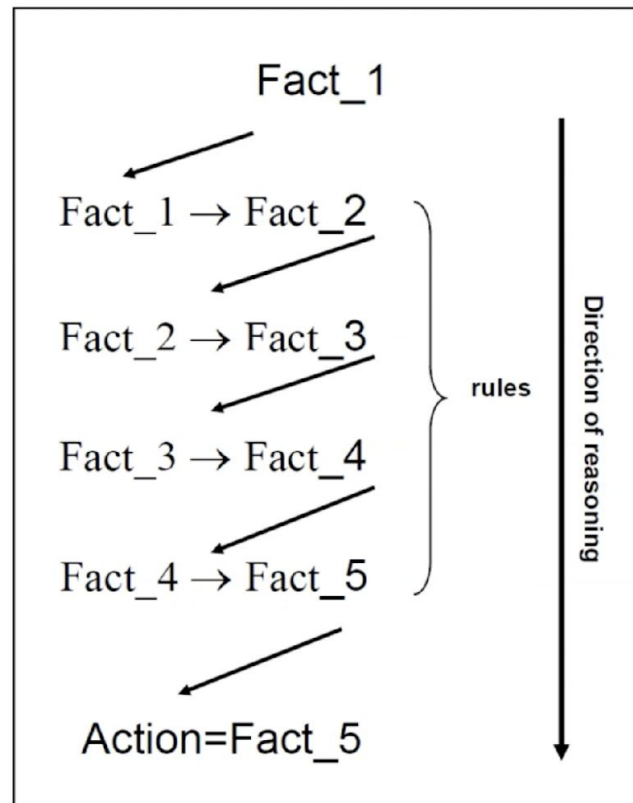


Repetir:

- Aplicar todas las reglas a los hechos actuales.
- Cada ejecución de una regla puede añadir nuevos hechos o activar nuevas reglas.

Continuar hasta que...

- No se añadan más hechos nuevos.



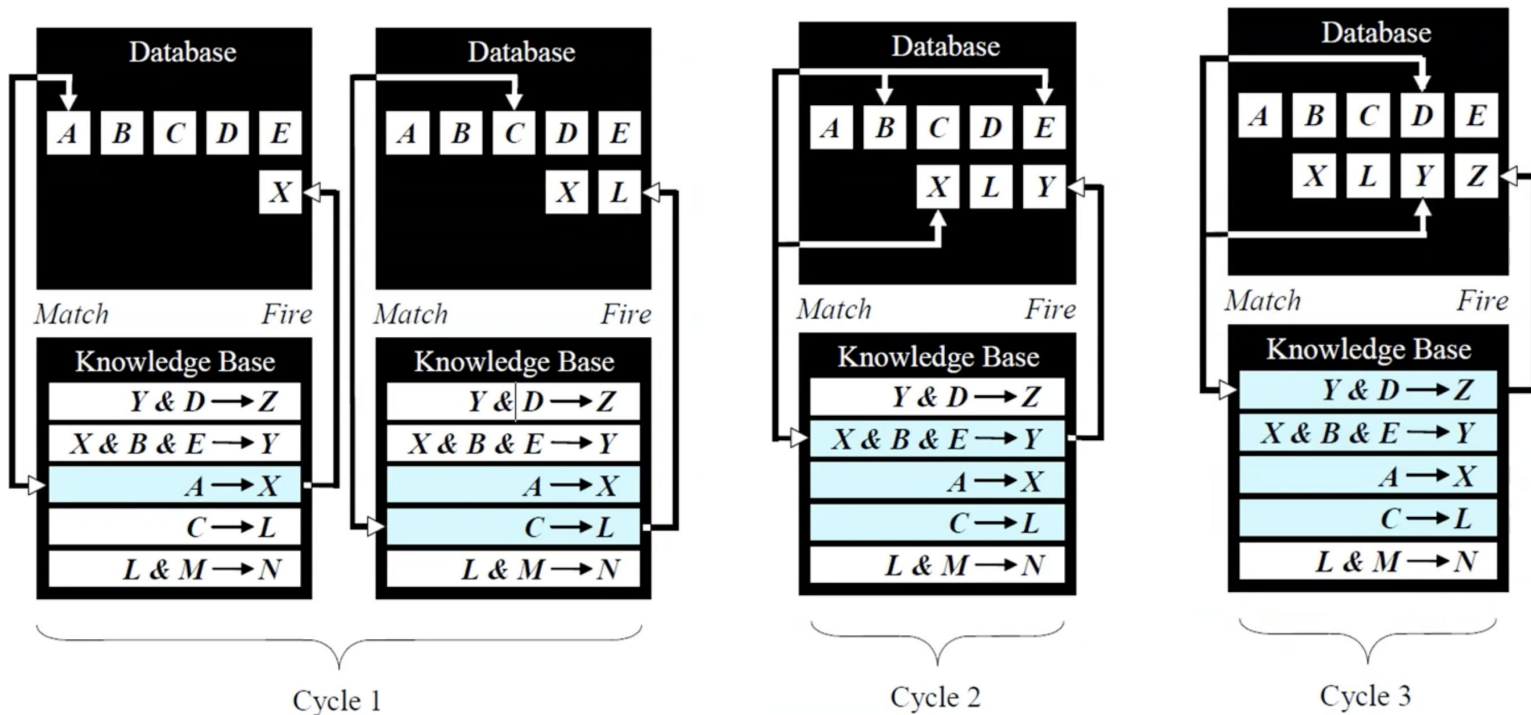


Encadenamiento hacia adelante con reglas



Por ejemplo, si
queremos
saber qué
ocurre cuando
ocurre: **A, B, C,**
D y E

Resulta en
Z válido



Reemplazar: "Knowledge base" por Rule Base (Reglas) y "Database" por Fact Base (Hechos).



Encadenamiento hacia adelante con reglas



El encadenamiento hacia adelante termina siendo una **cadena de inferencias**.

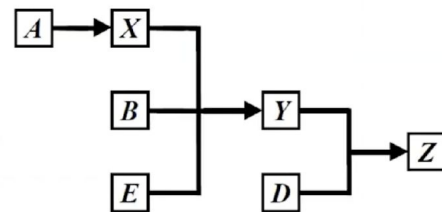
Ejemplo (anterior):

Supongamos que A, B y E son hechos.

Rule 1: IF Y is true
AND D is true
THEN Z is true

Rule 2: IF X is true
AND B is true
AND E is true
THEN Y is true

Rule 3: IF A is true
THEN X is true





Encadenamiento hacia atrás con reglas



Primero:

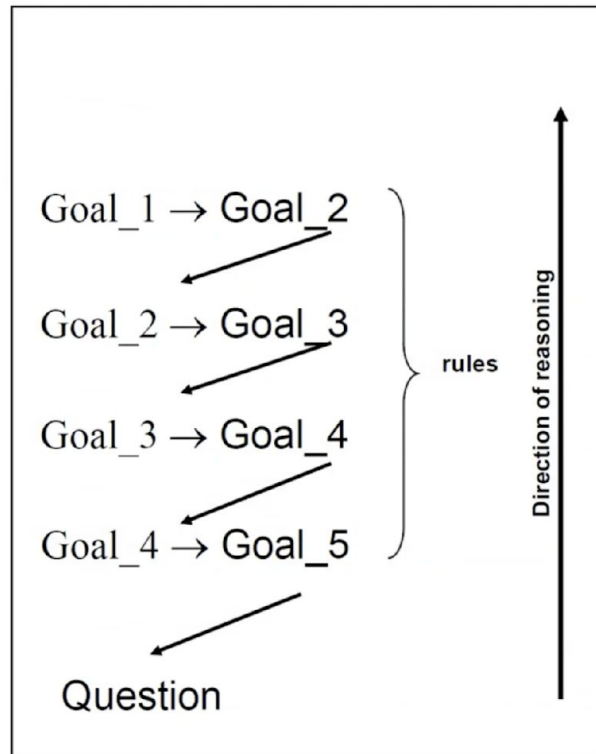
- Verificar si el consecuente (pregunta/objetivo inicial) está en la base de hechos.

Si no está:

- Buscar reglas que tengan ese consecuente como resultado.
- Los antecedentes de la regla se convierten en nuevos consecuentes objetivo (subobjetivos).

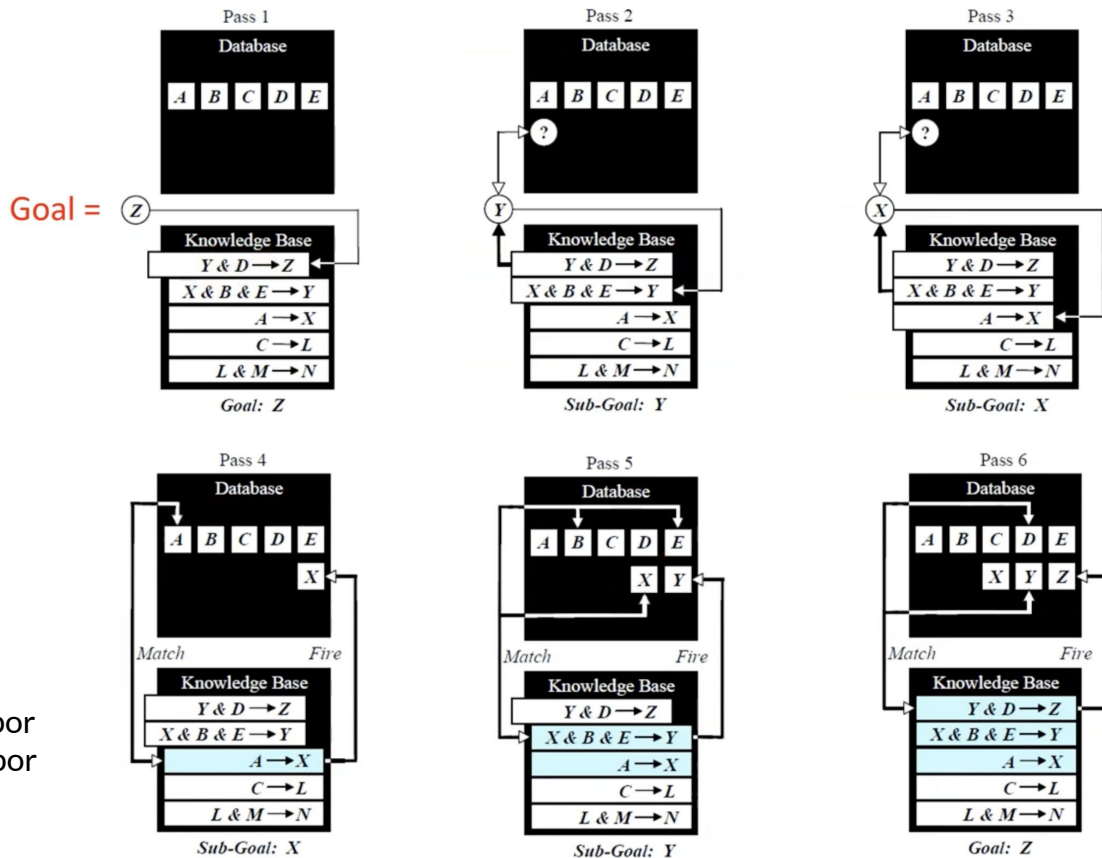
Continuar hasta que:

- Se satisfaga un conjunto conectado de subobjetivos con los hechos existentes (éxito).





Encadenamiento hacia atrás con reglas



Reemplazar: "Knowledge base" por Rule Base (Reglas) y "Database" por Fact Base (Hechos).

¡Comprobamos Z!



Fallo de una Consulta (Encadenamiento hacia atrás)



Interpretación del fallo como negación:

- Si no se puede demostrar una consulta, se asume que no es verdadera.

Sin embargo...

- También podría significar que la base de datos está incompleta o mal mantenida.
- En algunos casos, intentar recuperar información faltante puede llevar a bucles infinitos.

Solución en estos casos:

- Aplicar criterios de terminación que detecten esta situación y permitan que el sistema falle de manera controlada.



Encadenamiento mixto



Algunos Sistemas Expertos (ES) pueden **aplicar encadenamiento hacia adelante y hacia atrás dentro de la misma consulta.**

El sistema encadena en una dirección y luego cambia a la otra.

Ejemplo: Supongamos que tenemos un sistema experto que diagnostica enfermedades basadas en síntomas. Tenemos un paciente con fiebre y tos.

Encadenamiento Hacia Adelante

- La regla *posible-gripe* infiere automáticamente “*posible gripe*” si se detectan *fiebre* y *tos*.
- El hecho nuevo “*posible gripe*” dispara la regla *confirmar-gripe*.

Encadenamiento Hacia Atrás

- La regla *confirmar-gripe* pregunta al usuario si tiene dolor muscular.
- Si el usuario responde “*sí*”, **se confirma el diagnóstico de *gripe*.**



Eficiencia del motor de inferencia



¿Cómo reducir el costo de la coincidencia (matching)?

Algoritmos de Markov:

- Se aplican primero las reglas con mayor prioridad.
- Permite una ejecución más eficiente en sistemas de producción.
- Aún no es suficientemente eficiente para sistemas expertos con conjuntos de reglas muy grandes.

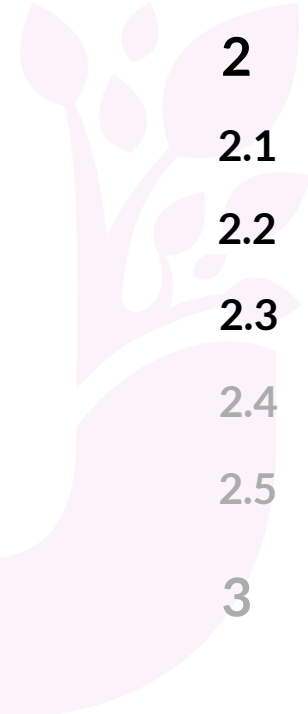
Guardar información intermedia de coincidencias – Algoritmo RETE:

- Comparte información intermedia de coincidencias entre reglas.
- Recalcula solo la información intermedia cuando hay cambios.
- Requiere memoria adicional para almacenar la información intermedia.
- Escala bien para conjuntos de reglas grandes.

Recalcular coincidencias solo para reglas afectadas – Algoritmo TREAT:

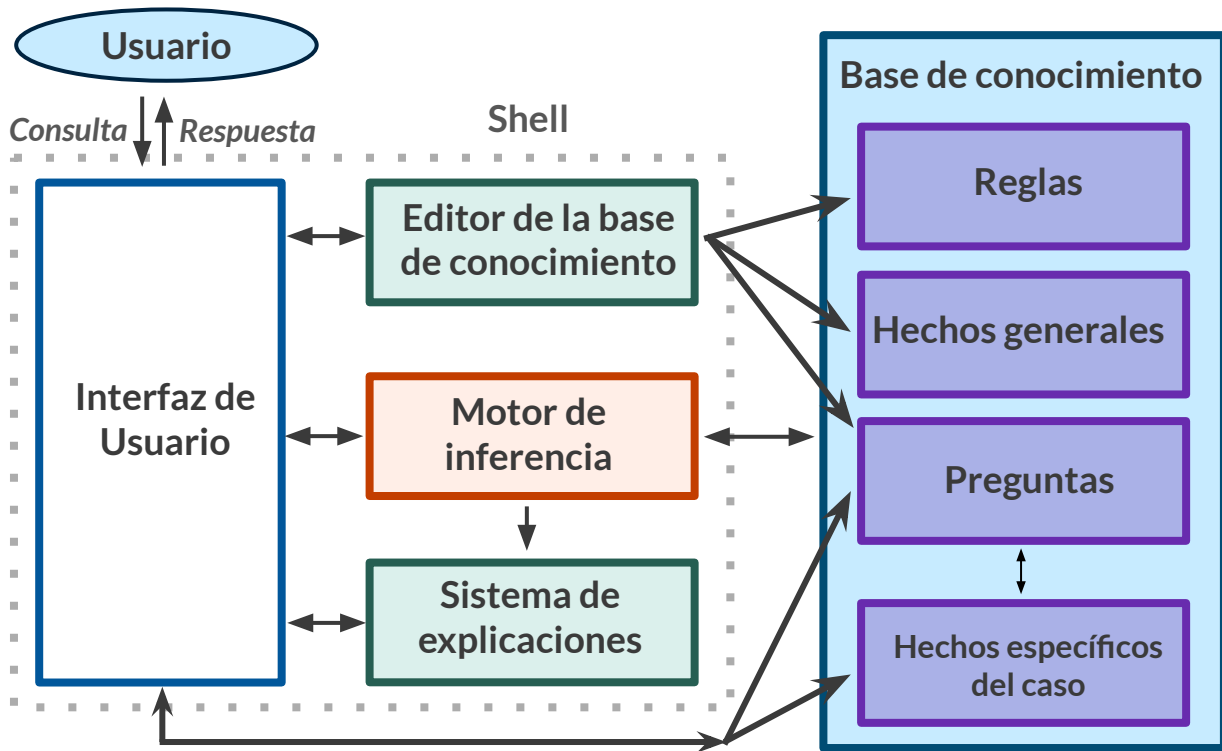
- Verifica cambios solo en las reglas en conflicto.
- Usa menos memoria que RETE.
- No escala tan bien para conjuntos de reglas muy grandes.

Temas de la clase

- 
- 1 Representación del conocimiento (*continuación*)
 - 2 **Introducción a Sistemas Expertos**
 - 2.1 Base de conocimientos (RBES)
 - 2.2 Motor de inferencia (RBES)
 - 2.3 Otros componentes
 - 2.4 Otros tipos de Sistemas Expertos
 - 2.5 Shells
 - 3 Pautas para el Trabajo Práctico



Interfaz de usuario



RBES = Rule Based Expert System



Interfaz de usuario



Medio de interacción entre un usuario que busca una solución y un Sistema Experto (ES), capaz de:

- Consultar el sistema.
- Responder preguntas.
- Editar hechos y reglas.
- Recibir soluciones y explicaciones.

La IU está conectada directa o indirectamente con todas las partes del Sistema Experto.

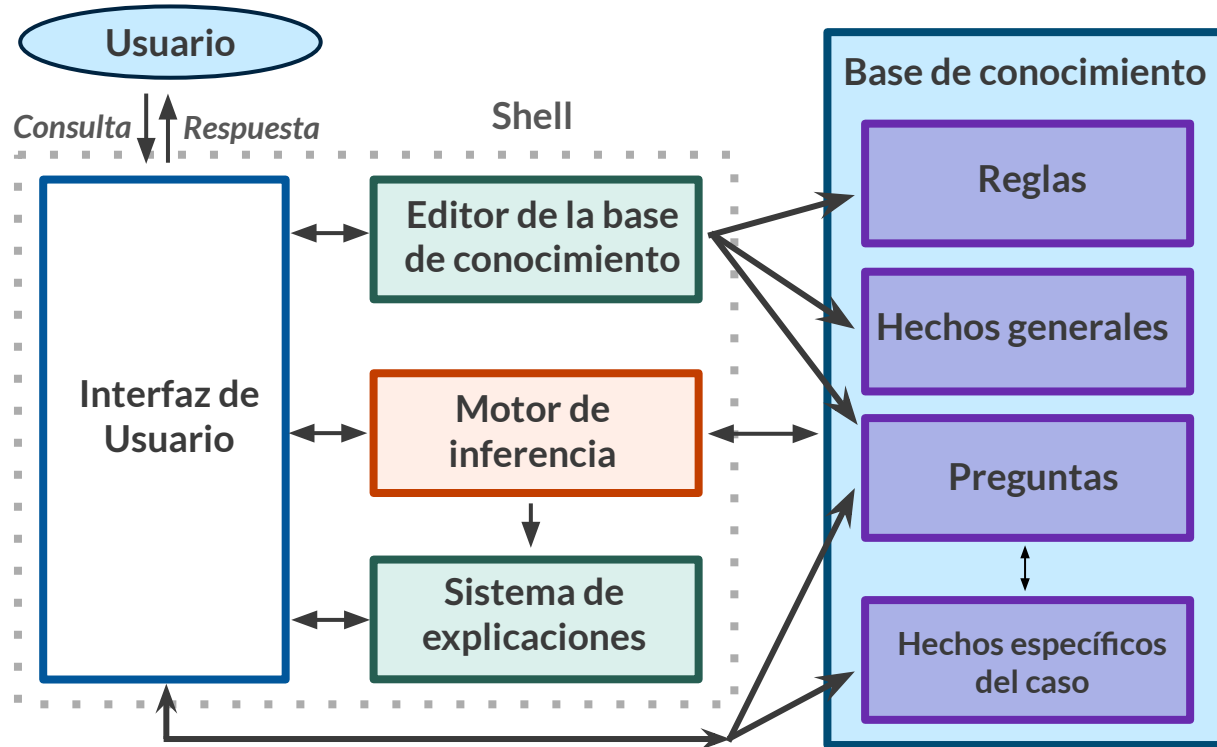
Tipos de IU en Sistemas Expertos

- Interacción basada en preguntas y respuestas.
- Menús interactivos (ejemplo: Interfaz Gráfica de Usuario - GUI).
- Procesamiento de Lenguaje Natural (NLP), permitiendo una interacción más intuitiva.

Debe hacer que el sistema sea fácil de usar para personas familiarizadas con el área de aplicación, pero que no necesariamente tengan habilidades de programación.



Editor de la base de conocimiento



RBES = Rule Based Expert System



Editor de la base de conocimiento



Editor de la base de conocimiento

También conocido como: Herramienta de adquisición de conocimiento.

Función principal:

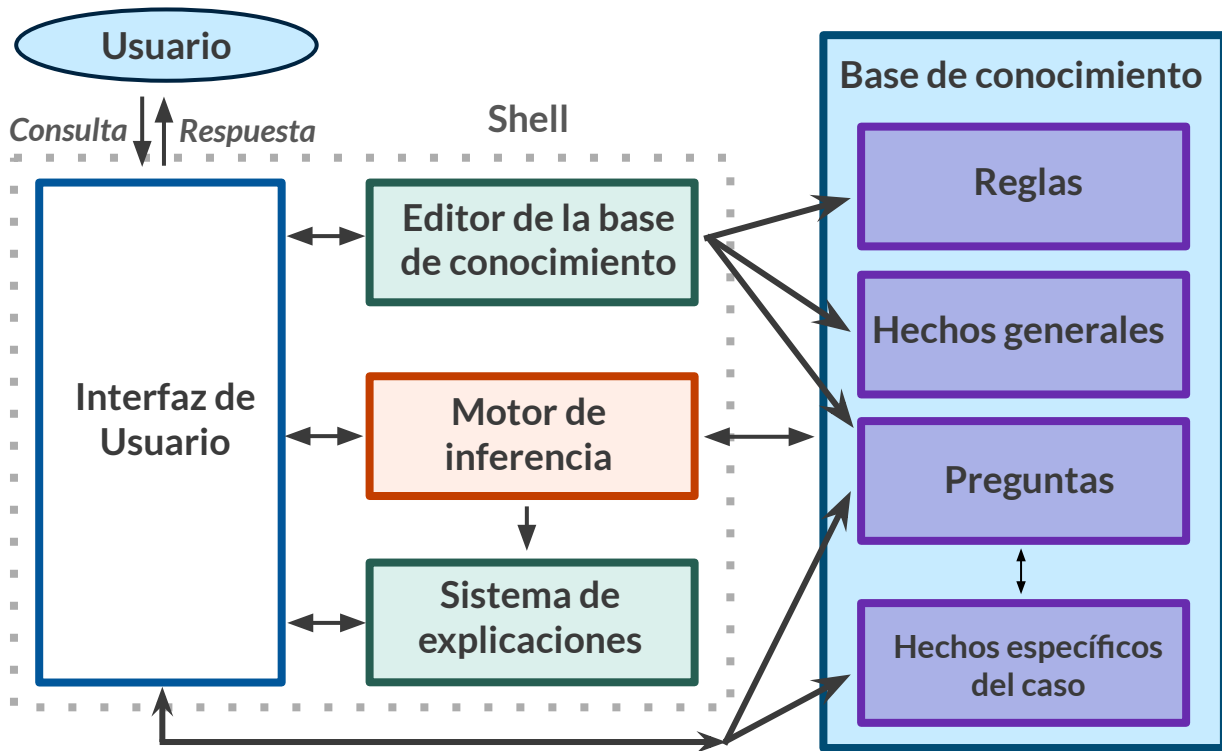
- Permite al usuario ingresar conocimiento automáticamente en el sistema, sin necesidad de saber programación.

Facilita:

- Integración de nuevo conocimiento.
- Edición del conocimiento existente.



Sistema de explicaciones



RBES = Rule Based Expert System



Sistema de explicaciones

También conocido como: Justificador (Justifier).

Función principal:

- Explica el razonamiento del Sistema Experto al usuario.

Responde preguntas como:

- ¿Cómo se llegó a una conclusión? (Cadena de razonamiento).
- ¿Qué reglas se activaron y qué hechos se dedujeron?
- ¿Por qué no se llegó a otra conclusión?
- ¿Por qué se necesita un hecho específico?
- ¿Por qué un hecho no fue utilizado?



Sistema de explicaciones

Ejemplo:

- Un médico necesita comprender el razonamiento detrás de una recomendación del sistema.

Ventaja clave de los Sistemas Expertos (ES) sobre las Redes Neuronales (NN):

- A pesar del éxito de los sistemas de redes neuronales, dada su explicabilidad, los Sistemas Expertos seguirán siendo clave en ciertas aplicaciones.

Usos clave de los Sistemas Expertos:

- Aplicaciones críticas para la seguridad, como diagnóstico médico.
- Donde sea legalmente obligatorio justificar decisiones con una explicación verbal.

Temas de la clase

- 
- 1 Representación del conocimiento (*continuación*)
 - 2 Introducción a Sistemas Expertos
 - 2.1 Base de conocimientos (RBES)
 - 2.2 Motor de inferencia (RBES)
 - 2.3 Otros componentes
 - 2.4 **Otros tipos de Sistemas Expertos**
 - 2.5 Shells
 - 3 Pautas para el Trabajo Práctico



Otros tipos de Sistemas Expertos



Sistemas Expertos **Difusos** (Fuzzy ES)

- Aplican lógica difusa (Fuzzy Logic) con reglas

Sistemas Expertos **basados en Marcos** (Frame-based ES)

- Usan representación por marcos (frame representation)

Sistemas Expertos **Híbridos** (Hybrid ES)

- Sistemas Expertos Neuronales (Neural ES)
- Reglas + Redes Neuronales

Sistemas Expertos **Neuronales Difusos** (Neural Fuzzy ES)

- Reglas difusas (Fuzzy Rules) + Redes Neuronales



Otros tipos de Sistemas Expertos



Sistemas Expertos **Difusos** (Fuzzy ES)

- Aplican lógica difusa (Fuzzy Logic) con reglas

Sistemas Expertos **basados en Marcos** (Frame-based ES)

- Usan representación por marcos (frame representation)

Sistemas Expertos **Híbridos** (Hybrid ES)

- Sistemas Expertos Neuronales (Neural ES)
- Reglas + Redes Neuronales

Sistemas Expertos **Neuronales Difusos** (Neural Fuzzy ES)

- Reglas difusas (Fuzzy Rules) + Redes Neuronales

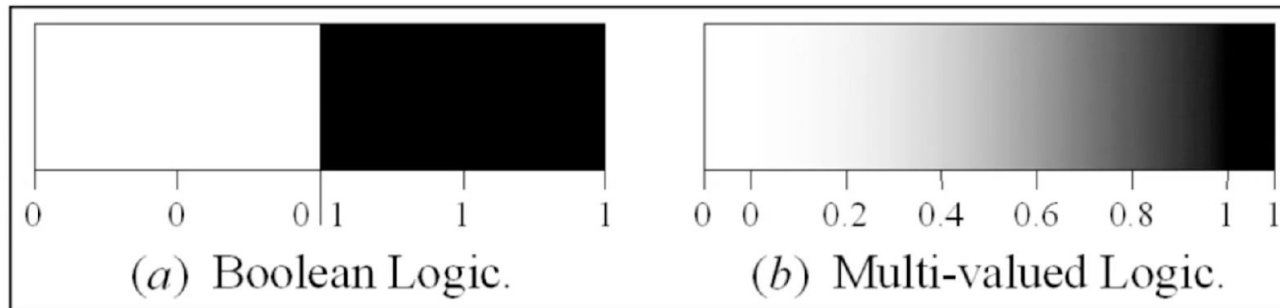


“Lógica que describe lo difuso” es decir, describe la vaguedad

Las decisiones se basan en un “**grado de pertenencia**” a un conjunto difuso en lugar de un valor Booleano (sí/no)

- Todas las cosas se describen en una escala gradual

¡No es probabilística!

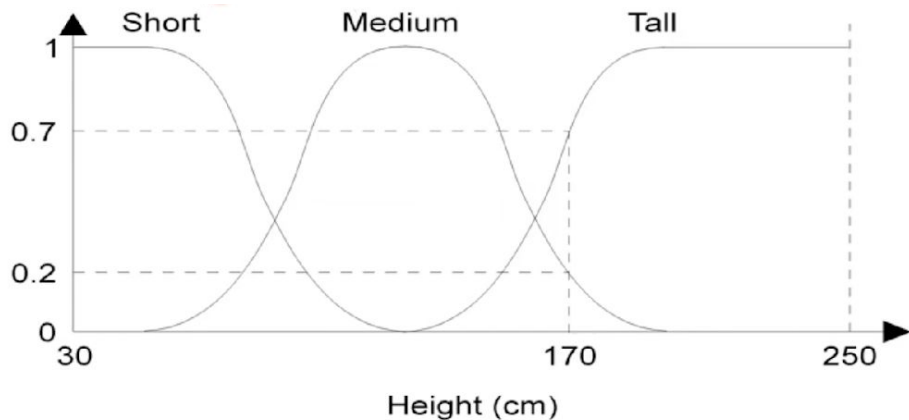




Conjuntos difusos



Cada valor de término difuso del antecedente (IF) o consecuente (THEN) de una regla puede ser representado por un conjunto difuso (fuzzy set).





Un ventilador Inteligente...

Regla clásica IF:THEN (lógica binaria)

Regla 1

- IF la temperatura > 30
- THEN (entonces) la velocidad del ventilador es 5

Regla 2

- IF la temperatura < 10
- THEN la velocidad del ventilador es 1

Regla IF:THEN (lógica difusa)

Regla 1

- IF la temperatura es caliente
- THEN la velocidad del ventilador es rápida

Regla 2

- IF la temperatura es baja
- THEN la velocidad del ventilador es baja



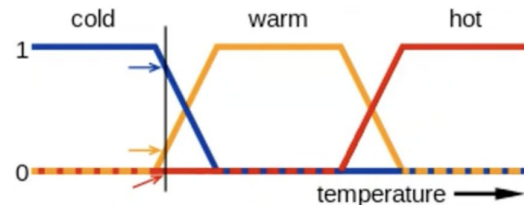
Mapeo de una entrada dada a una salida utilizando conjuntos difusos (4 pasos)

1 Fuzzificación:

- Convierte valores numéricos de entrada (*crisp*) en grados de pertenencia a conjuntos difusos mediante funciones de membresía.

2 Evaluación de Reglas:

- Tomar las entradas fuzzificadas (antecedentes) y aplicarle las reglas obteniendo salidas (consecuentes) difusas.



3 Agregación:

- Se combinan las salidas difusas de todas las reglas activadas en una única salida difusa global (curva compuesta).

4 Defuzzificación:

- Transformar la salida final en un valor preciso (*crisp*).

Boolean	Fuzzy
AND(x,y)	MIN(x,y)
OR(x,y)	MAX(x,y)
NOT(x)	$1 - x$



Inferencia difusa (ejemplo)



1. Fuzzificación:

28°C → 0.8 en "caliente", 0.2 en "muy caliente"

2. Evaluación de reglas:

Si caliente ENTONCES ventilador medio (fuerza 0.8)

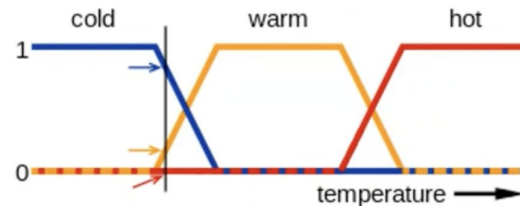
Si muy caliente ENTONCES ventilador alto (fuerza 0.2)

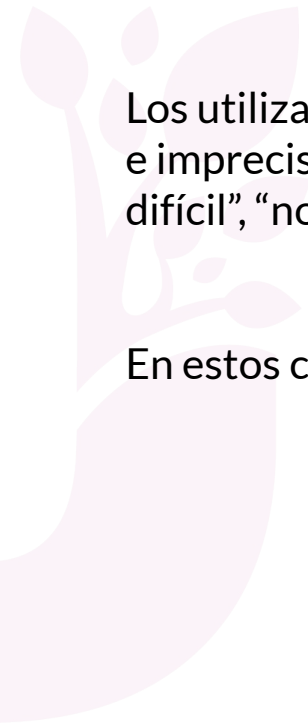
3. Agregación:

Combina conjuntos de "medio" y "alto" en un único conjunto (curva compuesta).

4. Defuzzificación:

Calcula centroide (por ej.) de la curva compuesta para asignar un valor de velocidad → 65% de potencia





Los utilizamos cuando queremos expresar conocimiento experto que utiliza palabras vagas e imprecisas como “ligeramente sobrecargado”, “fuertemente reducido”, “moderadamente difícil”, “no tan viejo”, “muy alto”.

En estos casos los sistemas expertos típicos basados en reglas no nos sirven.



Otros tipos de Sistemas Expertos



Sistemas Expertos **Difusos** (Fuzzy ES)

- Aplican lógica difusa (Fuzzy Logic) con reglas

Sistemas Expertos **basados en Marcos** (Frame-based ES)

- Usan representación por marcos (frame representation)

Sistemas Expertos **Híbridos** (Hybrid ES)

- Sistemas Expertos Neuronales (Neural ES)
- Reglas + Redes Neuronales

Sistemas Expertos **Neuronales Difusos** (Neural Fuzzy ES)

- Reglas difusas (Fuzzy Rules) + Redes Neuronales



Los **marcos** son una forma de representar el conocimiento mediante **estructuras** que agrupan información relacionada sobre un objeto, concepto o situación. **Cada marco es como una “plantilla” que contiene:**

- **Slots (Ranuras o atributos):** Representan propiedades o características.
- **Fillers (Valores o contenido):** Son los valores específicos de cada slot.

Se utilizan para modelar situaciones complejas donde un objeto tiene múltiples características.

Los marcos **tienen jerarquías**.

Ejemplo:

Un marco para el concepto “Coche” podría ser:

Nombre	
Coche	
Marca:	Toyota
Modelo:	Corolla
Color:	Rojo
Motor:	Híbrido
Slots	Fillers



Queremos representar el siguiente conocimiento:

1. **Cachita** es un **perro**.
2. Los **perros** tienen **cuatro patas**.
3. Cachita es de raza **Labrador** y tiene **5 años**.

Representación con Marcos:

- **Marco: Perro**
- Tipo: Mamífero
- Patas: Cuatro
- **Marco: Cachita (Instancia de Perro)**
- Es_un: Perro
- Raza: Labrador
- Edad: 5 años

Perro	
Tipo	Mamífero
Patatas	Cuatro



Cachita	
Es_un	Perro
Raza	Labrador
Edad	5 años



Objetivo: Determinar si **Cachita** tiene cuatro patas.

1. El marco **Cachita** indica que es un **Perro**.
2. El marco **Perro** tiene un **slot** que indica **Patas: Cuatro**.
3. Por **herencia de slots (según jerarquía)**, deducimos que **Cachita** tiene cuatro patas.

Conclusión: Cachita tiene cuatro patas.

Inferencias con Marcos

Perro	
Tipo	Mamífero
Patas	Cuatro



Cachita	
Es_un	Perro
Raza	Labrador
Edad	5 años



ES Basados en Marcos (Frame-Based ES)



Compuestos por:

- Marcos: almacenan el conocimiento
 - Métodos
 - Daemons
- } Operan con la información almacenada en los marcos realizando las inferencias

Métodos:

- Los methods son procedimientos (funciones) asociados a un marco o a uno de sus slots.
- Ejemplo: Slot: edad / Method: calcularEdad(fechaNacimiento)

Daemons:

- Los daemons son activadores automáticos: pequeñas rutinas que se ejecutan cuando ocurre algo con un slot, por ejemplo cuando se accede, se borra o se cambia su valor.
- Ej. If temperatura > 50 → activarAlarma()



Otros tipos de Sistemas Expertos



Sistemas Expertos **Difusos** (Fuzzy ES)

- Aplican lógica difusa (Fuzzy Logic) con reglas

Sistemas Expertos **basados en Marcos** (Frame-based ES)

- Usan representación por marcos (frame representation)

Sistemas Expertos **Híbridos** (Hybrid ES)

- Sistemas Expertos Neuronales (Neural ES)
- Reglas + Redes Neuronales

Sistemas Expertos **Neuronales Difusos** (Neural Fuzzy ES)

- Reglas difusas (Fuzzy Rules) + Redes Neuronales



Cualquier combinación de representaciones o integración de deducción con inducción. No todas las combinaciones funcionarán.

Tener en cuenta que:

- Los ES (Sistemas Expertos) implementan conocimiento humano existente.
- Pero este conocimiento aún puede tener errores.
- ¿Y si pudiéramos actualizar nuestro sistema mediante experiencia continua?
- Ahí entra el Aprendizaje Automático (Inductivo): entrenar un tomador de decisiones mediante ejemplos, en lugar de simplemente programarlo previamente.
- Potencial para superar incluso al experto humano.

Hibridación con sistemas sub-simbólicos, como redes neuronales:

- Podrían usarse para preprocesar datos (ej. imagen) y luego pasarlos a reglas.
- O para aprender mejores reglas.

Sistemas Híbridos: *Neural-ES*



- Se entrena una RN
- Las reglas se extraen de los pesos entre las distintas capas de las RN.
- Permite lidiar con datos ruidosos debido a la gran capacidad de generalizar de las RN.
- Una aproximación al razonamiento

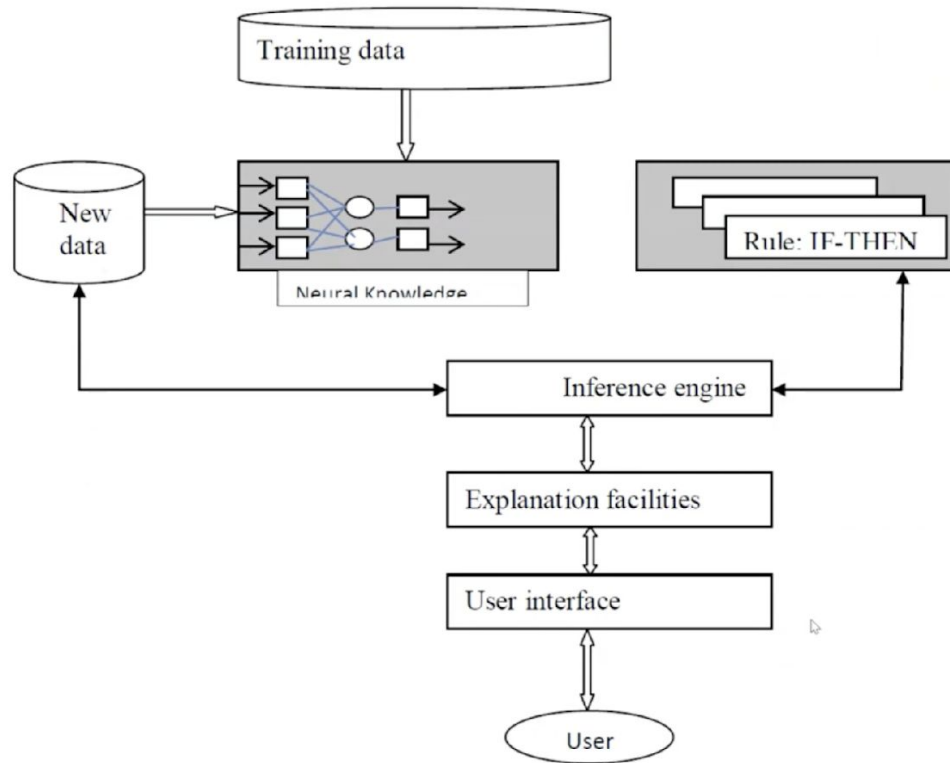


Figure5: Basic structure of neural expert system



Sistemas Híbridos: *Neuro-Fuzzy ES*



- Mientras que las redes neuronales (neural networks) son estructuras computacionales de bajo nivel que funcionan bien con datos brutos, la lógica difusa (fuzzy logic) maneja razonamientos de alto nivel utilizando información lingüística obtenida de expertos en el dominio.

Como resultado de la combinación:

- Las redes neuronales ganan transparencia.
- Los sistemas difusos (fuzzy systems) adquieren capacidad de aprendizaje.
- Cada capa en el sistema neuro-fuzzy corresponde a una etapa específica del proceso de inferencia difusa (fuzzy inference).



Comparación de Sistemas Expertos



Parámetros	Sistema experto basado en reglas	Sistema experto difuso (fuzzy)	Sistema experto basado en marcos (frames)	Sistema experto neuronal	Sistema experto neuro-difuso
Representación del conocimiento	Reglas IF-THEN en la base de conocimiento	Basado en el grado de pertenencia usando lógica difusa (fuzzy logic)	Conocimiento en marcos usando estructura jerárquica	Reglas IF-THEN en la base de conocimiento neuronal	Usa variables lingüísticas y reglas IF-THEN en una estructura difusa representada por redes neuronales como componente neuronal
Capacidad de aprendizaje	No puede aprender por sí mismo ni actualizar la base de conocimiento	No tiene capacidad para aprender de la experiencia	No puede aprender ni adaptarse a nuevos entornos	La red neuronal puede aprender, pero el proceso es una “caja negra” para el usuario	Tiene capacidad de aprendizaje mediante redes neuronales y reglas difusas
Tolerancia a la incertidumbre	Difícil de medir la incertidumbre	El razonamiento probabilístico puede manejar la incertidumbre	No es posible debido a la estructura del conocimiento	El razonamiento aproximado es posible	Usa scripts basados en razonamiento probabilístico
Tolerancia a la imprecisión	Muy baja, requiere información precisa	Alta, ya que la lógica difusa maneja la imprecisión	Muy baja. Datos imprecisos pueden generar errores	El componente neuronal puede trabajar con datos imprecisos	Muy alta, por la combinación de redes neuronales y lógica difusa
Facilidad de explicación	Sí, usando variables de explicación	Sí, usando variables de explicación	Sí, puede dar buenas explicaciones usando la salida	Sí, debido al componente difuso	Sí



Comparación de Sistemas Expertos (cont.)



Parámetros	Sistema experto basado en reglas	Sistema experto difuso (fuzzy)	Sistema experto basado en marcos (frames)	Sistema experto neuronal	Sistema experto neuro-difuso
Inferencia y toma de decisiones	Procesa reglas y deriva conclusiones	Procesa reglas usando fuzzificación y des-fuzzificación	Busca metas usando indexación y deamons	Red neuronal multicapa combinada con reglas de extracción	Sistema de inferencia difusa usando fuzzificación y des-fuzzificación
Actualización del conocimiento	Difícil agregar nuevas reglas	Difícil introducir nuevas variables lingüísticas en la estructura existente	No es posible	Sí. Con conocimiento externo, el componente neuronal puede aprender	Sí. Nuevas variables lingüísticas pueden añadirse fácilmente a la estructura existente
Mantenibilidad	Moderadamente difícil	Muy difícil	Fácil	Fácil	Difícil por el componente difuso
Adaptabilidad	No	No	Sí	Sí	Sí
Tiempo de procesamiento	Muy alto, ya que cada regla se procesa individualmente	Tiempo de procesamiento alto comparado con sistemas basados en reglas	El conocimiento en los marcos puede reutilizarse rápidamente	El aprendizaje inicial toma tiempo, pero luego el procesamiento es más rápido	El tiempo de aprendizaje es alto la primera vez. Una vez entrenado, el procesamiento se optimiza
Estructura del conocimiento	Ad-hoc, no permite comprender la dependencia lógica entre reglas	Bastante no estructurado	Altamente estructurado	Estructurado, almacenado en neuronas	Moderadamente estructurado

Temas de la clase

- 
- 1 Representación del conocimiento (*continuación*)
 - 2 Introducción a Sistemas Expertos
 - 2.1 Base de conocimientos (RBES)
 - 2.2 Motor de inferencia (RBES)
 - 2.3 Otros componentes
 - 2.4 Otros tipos de Sistemas Expertos
 - 2.5 Shells
 - 3 Pautas para el Trabajo Práctico



Pautas para el trabajo práctico



-  **Grupos:** los mismos que para el trabajo de investigación.
-  **Entregables:**
 - Notebook/repositorio con el desarrollo del Sistema Experto.
 - Vídeo 12min con la presentación que será posteado como material de la cátedra en el Moodle en este y los próximos cursos.
 - Opcional: IU en Streamlit o equivalente.
-  **Presentación presencial (04/06):** 12min de presentación más 3min de preguntas.
-  **Preguntas:** cada grupo tendrá asignado otro grupo (al menos uno) para hacerle preguntas (amistosas).
-  **Objetivo:** Construir un Sistema Experto basado en reglas que sea capaz resolver problemas de un dominio concreto.



Ejemplo 1: Recomendador de Plantas para Jardín

- **Descripción:** Un sistema que sugiere qué plantas son adecuadas según el clima, la cantidad de sol y el tipo de suelo.
- **Reglas:** Si el usuario vive en un clima seco y tiene poca agua disponible, recomendar suculentas o cactus.
- **Entrada:** Tipo de clima, exposición al sol, disponibilidad de agua.
- **Salida:** Lista de plantas recomendadas.



Ejemplo 2: Recomendador de Tipo de Ejercicio

- **Descripción:** Un sistema que sugiere qué tipo de ejercicio es adecuado según la edad, condición física y objetivos del usuario.
- **Reglas:** Si el usuario tiene problemas en las rodillas, recomendar natación; si quiere ganar masa muscular, sugerir levantamiento de pesas.
- **Entrada:** Edad, condición física, objetivos.
- **Salida:** Ejercicio/s recomendados.

—

¿Consultas?



—

¡Muchas gracias!

